

Data Lake, Data Warehouse & Lakehouse Architecture

Week 2 — Session 4 · Lab Manual

Lab 1 — Data Lake vs Data Warehouse vs Lakehouse

Business Scenario

"BrightCart," an online retailer, has two teams:

- The Finance team needs a dashboard every morning showing yesterday's revenue by region. They need it fast, accurate, and always in the same table format.
- The Data Science team wants to keep every raw file that ever comes in — order logs, app clickstreams, product images — even messy or incomplete ones, because they might use it for future ML models.

Think About It

- (a) Which storage approach fits Finance's need — lake, warehouse, or lakehouse? Why?

Ans: A data warehouse is most suitable for the finance team. Warehouse provides quick query capabilities, enforced schema to keep reporting error free and regular updates to keep data fresh

- (b) Which fits the Data Science team's need? Why?

Ans: A lakehouse is best for the DS team, as that provides possibility for queries but also keeps data as close to raw as possible

Multiple Choice Questions

MCQ 1.1 — A data warehouse is best described as:

- ☐ **A)** Cheap storage for any file type, queried later
- ☐ **B)** A structured repository with a fixed schema, optimized for fast business reporting
- ☐ **C)** A folder of random files with no organization
- ☐ **D)** Something used only for videos and images

ANS: B

MCQ 1.2 — Which statement is TRUE about a lakehouse?

- ☐ **A)** It's just a data lake renamed for marketing purposes
- ☐ **B)** It combines the low-cost flexible storage of a lake with the reliability and structure of a warehouse
- ☐ **C)** It can only be used for reporting, never for raw data
- ☐ **D)** It removes the need for any data cleaning

ANS: B

MCQ 1.3 — Scenario: A hospital wants to store years of unstructured MRI scan images cheaply, and separately run structured SQL reports on patient billing. What's the most sensible setup?

- ☐ **A)** One data warehouse for everything
- ☐ **B)** A data lake for MRI images, a data warehouse (or lakehouse) for billing reports
- ☐ **C)** A single spreadsheet
- ☐ **D)** No storage needed — just recompute every time

ANS: B

MCQ 1.4 — Scenario: A streaming platform wants one place where engineers can run SQL directly on raw viewing logs, get safe/reliable updates, but still keep storage costs low like a lake. What are they really asking for?

- ☐ **A)** A traditional on-premise data warehouse
- ☐ **B)** A lakehouse
- ☐ **C)** A single Excel workbook
- ☐ **D)** A backup tape system

ANS: B

MCQ 1.5 — Which of these is a genuine downside of a traditional data warehouse compared to a lake?

- ☐ **A)** It can't run SQL queries
- ☐ **B)** It's typically more expensive to scale and less flexible with unstructured/semi-structured data
- ☐ **C)** It never supports business reporting
- ☐ **D)** It can only store images

ANS: B

Lab 2 — Schema-on-Read vs Schema-on-Write

Business Scenario

"BrightCart" receives clickstream data from its app. Some events include a "page_name" field, some don't (older app versions didn't record it). The Data Science team wants to store this data immediately without rejecting anything. Meanwhile, Finance's revenue reports must always have exactly the same columns, correctly typed, or the dashboard breaks.

Think About It

- Which team benefits from "decide the structure later" (schema-on-read)?
ANS: DS/ML team
- Which team needs "structure decided before storage" (schema-on-write)?
ANS: Finance team
- What's the risk of using schema-on-read for Finance's revenue report?

ANS: Dashboards and reports might break, leading to the unnecessary consumption of time and resources

Multiple Choice Questions

MCQ 2.1 — Schema-on-read means:

- ☐ **A)** The structure/format is enforced before storing the data
- ☐ **B)** The structure is only figured out when someone actually queries/reads the data
- ☐ **C)** Data can never be read again once written
- ☐ **D)** Only applies to video files

ANS: B

MCQ 2.2 — A downside of schema-on-read is:

- ☐ **A)** It makes writing data slower
- ☐ **B)** Because nothing is validated at ingestion, messy or missing fields can cause problems later when someone tries to use the data
- ☐ **C)** It only works with spreadsheets
- ☐ **D)** It requires a data warehouse license

ANS: B

MCQ 2.3 — Scenario: A bank's transaction system must reject any incoming record missing an account number, before it's ever stored. This is an example of:

- ☐ **A)** Schema-on-read
- ☐ **B)** Schema-on-write
- ☐ **C)** No schema at all
- ☐ **D)** Unstructured storage

ANS: B

MCQ 2.4 — Scenario: A social media company ingests millions of posts per day in constantly-changing formats (new fields added by app updates every month). Enforcing a strict schema at write time would mean:

- ☐ **A)** Nothing changes, it's the same either way
- ☐ **B)** Frequent pipeline failures or rejected records every time the format changes slightly
- ☐ **C)** Faster ingestion with zero downsides
- ☐ **D)** The posts would automatically fix themselves

ANS: B

MCQ 2.5 — Which best describes the general trade-off between schema-on-read and schema-on-write?

- ☐ **A)** Schema-on-read = flexible ingestion but more work later; schema-on-write = more upfront effort but more reliable/consistent data later
- ☐ **B)** They are exactly the same, just different names
- ☐ **C)** Schema-on-write is always better in every situation
- ☐ **D)** Schema-on-read only works for numeric data

ANS: A

Lab 3 — The Bronze (Raw) Zone

Business Scenario

BrightCart's engineering team ingests raw order data every day exactly as it arrives from the checkout system — no cleaning, no filtering — into a "Bronze" folder, organized by the date it arrived.

Think About It

- Why would a company deliberately keep messy, unprocessed data instead of cleaning it immediately?
ANS: To have a data audit trail. To investigate errors and bugs. Check when and why pipelines break
- What happens if a bug is found in tomorrow's cleaning logic — how does having Bronze data help?
ANS: With bronze, the team could properly investigate the origin of the bug and how it could be robustly avoided.

Multiple Choice Questions

MCQ 3.1 — The Bronze (raw) zone typically contains:

- ☐ A) Only the last 24 hours of clean data
- ☐ B) Raw, unprocessed data as it arrived, often kept for a long time
- ☐ C) Only finished dashboard tables
- ☐ D) Data with all business rules already applied

ANS: B

MCQ 3.2 — Why organize Bronze data by date (e.g., year/month/day folders)?

- ☐ A) It's a legal requirement in every country
- ☐ B) It makes it easier to reprocess, audit, or re-run a specific day's data if something goes wrong
- ☐ C) It automatically removes duplicate records
- ☐ D) It makes the data unreadable to unauthorized users

ANS: B

MCQ 3.3 — Scenario: A retailer discovers 3 weeks later that a currency conversion bug corrupted last Tuesday's sales numbers in their dashboard. What allows them to fix this without asking customers to resend data?

- ☐ A) Nothing — the data is gone forever
- ☐ B) The raw Bronze data from that Tuesday is still there and can be reprocessed
- ☐ C) They must guess the correct numbers
- ☐ D) They restart the entire warehouse

ANS: B

MCQ 3.4 — Scenario: A compliance officer asks, "Can you show me exactly what data we received from the payment provider on March 5th, before any of our own processing touched it?" Which zone answers this?

- ☐ A) Gold
- ☐ B) Bronze
- ☐ C) A dashboard
- ☐ D) None — this information isn't kept anywhere

ANS: B

MCQ 3.5 — Which of these is a valid reason NOT to skip the Bronze zone and jump straight to cleaning data on arrival?

- ☐ A) Skipping Bronze saves no time at all

- ☐ **B)** Without Bronze, you lose the ability to redo cleaning/transformation later if requirements or bugs are found
- ☐ **C)** Bronze zones are required by every programming language
- ☐ **D)** There is no real reason, skipping it is always fine

ANS: B

Lab 4 — The Silver (Cleaned) Zone

Business Scenario

BrightCart's engineering team takes the raw Bronze order data and removes duplicates, fixes data types (e.g., "10" the text becomes 10 the number), and standardizes column names. This cleaned version is stored separately as "Silver," and multiple teams (Finance, Marketing, Data Science) all reuse it instead of re-cleaning the raw data themselves.

Think About It

- Why is it more efficient for multiple teams to share one Silver dataset instead of each team cleaning the raw data on their own?

ANS: For large bronze storage, multiple query and processing is resource wasteful. Having one extensive silver table where different users can query and aggregate from mitigates this

- Is Silver data usually the final dataset shown on a dashboard? Why or why not?

ANS: No. Silver is quite extensive and does not pertain to specific users. Usually don't contain aggregates

Multiple Choice Questions

MCQ 4.1 — Compared to raw Bronze data, Silver data is:

- ☐ **A)** Exactly identical, just renamed
- ☐ **B)** Parsed, validated, and deduplicated — a reusable, trustworthy intermediate version
- ☐ **C)** Only used for storing images
- ☐ **D)** Always deleted after 24 hours

ANS: B

MCQ 4.2 — Silver-zone data is best described as:

- ☐ **A)** The final dataset business executives view directly on dashboards
- ☐ **B)** A clean, validated dataset — still an intermediate step before final business-ready tables
- ☐ **C)** The same thing as the Bronze zone
- ☐ **D)** Data that only data scientists are allowed to see

ANS: B

MCQ 4.3 — Scenario: Three different teams keep writing their own scripts to clean the same messy raw order data, each producing slightly different results. What architectural fix solves this?

- ☐ **A)** Delete the raw data
- ☐ **B)** Build one shared Silver dataset that everyone reuses
- ☐ **C)** Give up on cleaning entirely
- ☐ **D)** Only let one team ever look at the data

ANS: B

MCQ 4.4 — Scenario: During cleaning, the Silver process removes exact duplicate order records that appeared twice due to a network retry. This step is an example of:

- ☐ **A)** Data aggregation for reporting
- ☐ **B)** Data validation/deduplication, a typical Silver-zone task
- ☐ **C)** Something that should only happen in Gold
- ☐ **D)** Something that should only happen in Bronze

ANS: B

MCQ 4.5 — Which of these tasks would you expect to happen in Silver, NOT in Bronze?

- ☐ **A)** Storing the file exactly as it arrived
- ☐ **B)** Converting a text field like "20.5" into an actual numeric value
- ☐ **C)** Keeping every historical raw file forever
- ☐ **D)** Ingesting data with zero changes

ANS: B

Lab 5 — The Gold (Curated) Zone

Business Scenario

BrightCart's Finance team needs a single, ready-to-use table: total sales per region per day, refreshed every morning, that plugs directly into their dashboard tool.

Think About It

- Why would the company build a separate "Gold" table instead of having Finance query the raw Bronze data directly every morning?

ANS: Personalized data for specific teams. No need for constant reprocessing of bronze.

- What business value does a Gold table provide that Silver doesn't?

ANS: Silver just contains clean information. Gold goes one step further to get personalized insights and indicators from the information

Multiple Choice Questions

MCQ 5.1 — Gold-zone tables are typically:

- ☐ **A)** Raw and unprocessed
- ☐ **B)** Curated, aggregated, and business-ready — optimized for dashboards and reporting
- ☐ **C)** Only readable by engineers

- ☐ **D)** Rebuilt from scratch by hand every day

ANS: B

MCQ 5.2 — The simplest reason a BI dashboard tool (like Power BI, Tableau, or Excel) connects to Gold tables instead of raw Bronze files is:

- ☐ **A)** Gold data is already clean, aggregated, and in a consistent format ready for reporting
- ☐ **B)** Bronze files are usually password protected
- ☐ **C)** Gold tables are always smaller in storage size
- ☐ **D)** There's no real reason, it's arbitrary

ANS: A

MCQ 5.3 — Scenario: The CEO asks for "total revenue by region, updated daily" on a live dashboard. Which zone should power that dashboard?

- ☐ **A)** Bronze
- ☐ **B)** Silver
- ☐ **C)** Gold
- ☐ **D)** None — recompute from scratch each time from source systems

ANS: C

MCQ 5.4 — Scenario: A retail chain builds a Gold table called "fact_daily_store_sales" that joins cleaned sales data with store location and product category info, pre-aggregated by day. This is a good example of Gold because it is:

- ☐ **A)** Raw and untouched
- ☐ **B)** Business-friendly, pre-joined, and ready for fast reporting without extra work
- ☐ **C)** Something only engineers would ever look at
- ☐ **D)** A duplicate copy of Bronze

ANS: B

MCQ 5.5 — Which statement about Gold tables is FALSE?

- ☐ **A)** They are usually refreshed on a schedule (e.g., daily/hourly)
- ☐ **B)** They are meant to be queried directly by business users and dashboards
- ☐ **C)** They should always be rebuilt entirely from scratch by hand every time someone needs a number
- ☐ **D)** They are typically smaller and more aggregated than Silver

ANS: C

Lab 6 — Storage-First & Separating Compute from Storage

Business Scenario

BrightCart stores all its Bronze/Silver/Gold data in one central cloud storage location. Three different tools access this same data: the Finance team's SQL reporting tool, the Data Science team's Python notebooks, and an external auditor's reporting software.

Think About It

- Why can all three tools read the same data without needing their own separate copies or stepping on each other?

ANS: Because of the different processing steps leading to different storage layers, the data can be assessed with different tools depending on the business needs.

- What would be the downside if data could only be accessed through one specific tool/database engine?

ANS: Overloaded servers. Long processing for information that need to be delivered quickly. Loss of information over time .

Multiple Choice Questions

MCQ 6.1 — Separating "storage" (where data lives) from "compute" (the tools that process it) mainly allows:

- ☐ **A)** Data to disappear automatically after use
- ☐ **B)** Multiple tools/teams to access the same data independently, and scale processing power only when needed
- ☐ **C)** Guaranteed zero-latency access at all times
- ☐ **D)** Elimination of any need for security

ANS: B

MCQ 6.2 — Scenario: BrightCart's Marketing team has a huge one-time analysis to run this week, needing 10x more processing power than usual, but only for 3 days. With compute/storage separation, what's the sensible approach?

- ☐ **A)** Buy a new permanent server for 10x capacity
- ☐ **B)** Temporarily scale up compute power for those 3 days, then scale back down — storage stays untouched
- ☐ **C)** Copy all the data to a new physical location
- ☐ **D)** Refuse the request — it's not technically possible

ANS: B

MCQ 6.3 — In a traditional tightly-coupled database (compute and storage bundled together), what typically happens if you need more query power?

- ☐ **A)** Nothing changes, it scales itself infinitely for free
- ☐ **B)** You often have to scale up the entire machine (including storage you didn't need more of), which is costly and less flexible
- ☐ **C)** You simply add more storage disks and query speed increases automatically
- ☐ **D)** It's impossible to add more power at all

ANS: B

MCQ 6.4 — Scenario: BrightCart shuts down its Python analysis jobs for the weekend, but the SQL reporting tool still needs 24/7 access to the same Gold tables. Storage-first architecture allows this because:

- ☐ **A)** The data itself doesn't depend on any single compute engine being on
- ☐ **B)** Data disappears when compute is off
- ☐ **C)** Only one tool can ever exist at a time
- ☐ **D)** SQL tools cannot read shared storage

ANS: A

Lab 7 — Metadata & Data Catalogs

Business Scenario

A new analyst joins BrightCart and asks: "Where is the sales data? Who owns it? What does each column actually mean? Has it changed recently?" Without documentation, they'd have to ask around or dig through files manually.

Think About It

- What problems arise in a company if there's no catalog or documentation describing available datasets?

ANS: There then is an unnecessary and redundant task of having to explain the data everytime. The could be cumbersome for large datasets

- How does a catalog help with trust and governance, not just findability?

ANS: With catalogs consistency is handled better. Audits don't always require full query but just look ups in catalogs and metadata to know what changed.

Multiple Choice Questions

MCQ 7.1 — The main purpose of a data catalog is to:

- ☐ **A)** Physically store the data files
- ☐ **B)** Help people discover what data exists, who owns it, what it means, and where it came from
- ☐ **C)** Replace the need for any data storage
- ☐ **D)** Make data load faster over the internet

ANS: B

MCQ 7.2 — Scenario: An auditor needs to know exactly when a "revenue" column's calculation logic changed last year. A good catalog/metadata system should be able to show:

- ☐ **A)** Nothing — this isn't possible
- ☐ **B)** The history/lineage of changes made to that dataset over time
- ☐ **C)** Only the current value, never past changes
- ☐ **D)** The auditor's own notes, not the system's

ANS: B

MCQ 7.3 — Which of these is NOT typically part of a data catalog entry?

- ☐ **A)** Table owner
- ☐ **B)** Column descriptions
- ☐ **C)** The literal raw bytes of every file, duplicated
- ☐ **D)** Data lineage (where it came from, what transformed it)

ANS: B

MCQ 7.4 — Scenario: Two teams both have a table called "customers" with different meanings — one is "signed-up users," the other is "paying customers only." Without a catalog, what's the likely business risk?

- ☐ **A)** None, the names being the same causes no confusion
- ☐ **B)** Reports or decisions could be based on the wrong definition of "customers," leading to incorrect conclusions
- ☐ **C)** The database would refuse to run at all
- ☐ **D)** Both tables would automatically merge correctly

ANS: B

MCQ 7.5 — A well-maintained data catalog mainly builds:

- ☐ **A)** Faster internet speeds
- ☐ **B)** Trust and shared understanding of data across teams
- ☐ **C)** More raw storage space
- ☐ **D)** Automatic dashboards with no design needed

ANS: B

Capstone Scenario — Design the Full Pipeline (No Coding)

Business Scenario

"BrightCart" is launching a new mobile app and wants to track user activity (clicks, screen views, purchases) end-to-end — from raw ingestion to an executive dashboard.

Task

On paper (or in this document), sketch your answer to each:

1. Where would raw app event data land first, and why (name the zone)?

The raw data would land in a bronze dome. Here no processing is done on the data. The data is written once and if everything works perfectly, should be read minimally (for the silver layer and audit)

2. What cleaning/validation steps would move it to the next zone?

1. Schema enforcement
2. Type parsing
3. Deduplication
4. Normalisation
5. Handling missing values

3. What final business-ready table would you build, and who would use it?

Final business table would be a Gold table aggregating information like

1. What products do users show more interest in and need more information on
2. What products do users abandon
3. Temporal user behaviours

This data is suitable for

1. Product development teams
2. Finance teams
3. Procurement teams

4. If the marketing team later finds a bug in how "purchase" events were being counted, which zone would you fix and re-run from — and why does keeping that earlier zone matter?

The silver zone would be recomputed with more accurate logic from the bronze zone. Keeping the bronze zone ensures that in the case of change in business logic, the information is not lost over time

5. Would you use a data lake, warehouse, or lakehouse approach overall for this app? Justify your choice in 2–3 sentences.

Lake house. Clickstreams can be semi structured as user behaviours can vary. However it's still necessary to easily query the data
